```
@assembly = @part.assemblies.find(1)
```

4.4.1.13 collection.where(...)

The `collection.where` method finds objects within the collection based on the conditions supplied but the objects are loaded lazily meaning that the database is queried only when the object(s) are accessed. It also adds the additional condition that the object must be in the collection.

```
@new_assemblies = @part.assemblies.where("created_at > ?", 2.days.ago)
```

4.4.1.14 collection.exists?(...)

The `collection.exists?` method checks whether an object meeting the supplied conditions exists in the collection. It uses the same syntax and options as `ActiveRecord::Base.exists?`.

4.4.1.15 collection.build(attributes = {})

The `collection.build` method returns a new object of the associated type. This object will be instantiated from the passed attributes, and the link through the join table will be created, but the associated object will *not* yet be saved.

```
@assembly = @part.assemblies.build({assembly_name: "Transmission
housing"})
```

4.4.1.16 collection.create(attributes = {})

The `collection.create` method returns a new object of the associated type. This object will be instantiated from the passed attributes, the link through the join table will be created, and, once it passes all of the validations specified on the associated model, the associated object *will* be saved.

```
@assembly = @part.assemblies.create({assembly_name: "Transmission
housing"})
```

4.4.1.17 collection.create!(attributes = {})

Does the same as `collection.create`, but raises `ActiveRecord::RecordInvalid` if the record is invalid.

4.4.2 Options for has_and_belongs_to_many

While Rails uses intelligent defaults that will work well in most situations, there may be times when you want to customize the behavior of the `has_and_belongs_to_many` association reference. Such customizations can easily be accomplished by passing options when you create the association. For example, this association uses two such options:

```
class Parts < ActiveRecord::Base
  has_and_belongs_to_many :assemblies, -> { readonly },
                                       autosave: true
end
```

The `has_and_belongs_to_many` association supports these options:

- `:association_foreign_key`
- `:autosave`
- `:class_name`
- `:foreign_key`
- `:join_table`
- `:validate`

4.4.2.1 :association_foreign_key

By convention, Rails assumes that the column in the join table used to hold the foreign key pointing to the other model is the name of that model with the suffix `_id` added. The `:association_foreign_key` option lets you set the name of the foreign key directly:

The `:foreign_key` and `:association_foreign_key` options are useful when setting up a many-to-many self-join. For example:

```
class User < ActiveRecord::Base
  has_and_belongs_to_many :friends,
      class_name: "User",
      foreign_key: "this_user_id",
      association_foreign_key: "other_user_id"
```